Lab4 - Assertions

Following our step-by-step verification of a simple processor (so far you already worked on adder, ALU, and register file!) for this lab session we will focus on the front-end of the pipeline: the fetch and decode units. And the aim of the session will focus on adding assertions to our testbench.

The lab session is divided into two sections: the first focuses on the fetch unit, while the second covers the decode unit. Unlike prior sessions, in this session the specification, design, and testbench will be provided for each unit. Your responsibility will be to develop assertions for each design, referencing the module's specification.

The assertions to be added to the fetch unit will be guided: you will find a sentence describing the aim of each assert, and you must provide the appropriate SV code for the assertion and add it into the provided testbench. On the other hand, for the decode unit it is your job to infer the most appropriate set of assertions, and like in previous module, add them to the given testbench.

1. Get Ready for this Lab

In the web of the course at https://docencia.ac.upc.edu/master/MIRI/PV/labs.html, for session Lab4, you will find a package front_end.tgz containing two directories with the following files:

a) fetch_unit.v and tb_fetch_unit.sv: corresponds to the design and testbench for the fetch unit module.
b) decode_unit.v and tb_decode_unit.sv: the equivalent for the decode unit module.

There is also a simple Makefile in each directory.

Testbenches provide simple, directed tests, which are sufficient for this lab session. If you think additional tests are needed to verify your assertions, you may include them.

2. Design #1: Fetch Unit

2.1. Specification

The fetch unit is responsible for supplying instructions to the pipeline. It maintains a program counter (PC), updates it according to control signals, and outputs the instruction stored at the current PC address. The unit supports sequential execution and branch redirection. Instruction memory is modeled internally as a simple array initialized with dummy values.

This is the interface of the module:

Signal Name	Direction	Width	Description
`clk`	Input	1	Rising-edge system clock. All sequential behavior is synchronous
`rst_n`	Input	1	Active-low reset. When asserted, PC is cleared to 0
`pc_en`	Input	1	Enables sequential PC increment
`branch_en`	Input	1	When asserted, PC is loaded with branch_addr instead of incrementing
`branch_addr`	Input	8	Target address for branch redirection
`instr`	Output	16	Instruction word fetched from memory at the current PC address

The fetch unit maintains an internal program counter (PC) that determines which instruction is supplied to the pipeline. On every rising edge of the clock, the PC is updated according to a strict priority of control signals. If reset is asserted, the PC is forced to zero regardless of any other inputs. If reset is not active and a branch request is signaled, the PC is immediately loaded with the branch target address. In the absence of a branch, if the sequential enable signal is asserted, the PC advances by 2, assuming 16-bit instructions and byte addressing. When neither branch nor sequential enable is active, the PC simply retains its previous value. The next PC value is computed combinationally based on control signals, ensuring that both the PC and instruction outputs are updated synchronously and consistently on each clock edge.

The instruction output reflects the contents of the memory location that will be addressed by the PC in the next cycle. This ensures that the instruction is fetched ahead of execution and remains consistent with the updated PC value. Memory is modeled as a 256-entry array of sixteen-bit words, each initialized with a dummy value equal to its address index. This initialization strategy allows for easy verification of PC behavior, as the instruction output directly reflects the PC value.

Branch requests take precedence over sequential increments, so if both signals are asserted in the same cycle, the branch target address is chosen. The PC must remain within the legal range of zero to 255 at all times, and any attempt to access outside this range is considered invalid behavior.

Together, these rules define the functional behavior of the fetch unit: reset initializes the PC to zero, branch requests redirect execution, sequential enable advances the PC, and the instruction output is always consistent with the memory contents at the current PC.

2.2. Testbench

The provided tb_fetch_unit.sv testbench can be used as a basis for you to add the assertions. It contains test generation and result checking. You do not need to implement any other functionality, only focus on adding the assertions.

2.3. Assertions

Please, work on the following assertions:

Assertion	Intent
1. Reset behavior	On reset, PC must go to 0
2. Branch priority	If **`branch_en`** is asserted, PC must load **`branch_addr`** regardless of **`pc_en`**
3. PC increment	If **`pc_en`** is asserted (and no branch), PC must increment by 2
4. PC stability	If neither **`branch_en`** not **`pc_e`**n is asserted, PC must hold its value
5. Instruction consistency	i**`nstr`** must always equal mem[PC]
6. PC range safety	PC must never go out of memory bounds (0-255)

Hint #1: Only use sequences when needed; recall they represent behavior over time.

Hint #2: You can use $past function to access value from previous cycle.

Hint #3: Recall: Non-blocking assignments (<=) in sequential logic update the value after the clock edge.

Hint #4: It can be helpful to start by running the testbench and examining the DUT's waveform for each test.

Perform the following tasks:

a) Try that all your assertions pass with the provided testbench and design.
b) Change the design (introduce a bug) and check that your assert/s fail.

3. Design #2: Decode Unit

3.1. Specification

The decode stage is a pipeline block that interprets 16 bit instructions received from the fetch stage. It extracts the opcode, destination register, source register, and immediate field, and prepares them for the execute stage. To model realistic pipeline behavior, the decode stage introduces a fixed two cycle latency between instruction acceptance and completion. It also performs simple hazard detection: if the destination register of the previous instruction matches the source register of the current one, the stage asserts a stall signal to delay acceptance until the hazard clears. The stage receives instructions from fetch via an **`instr_valid`** signal and signals completion with a single-cycle **`decode_done`** pulse.

This is the interface of the module:

Signal	Direction	Width	Description
`clk`	Input	1	Rising-edge system clock. All sequential behavior is synchronous
`rst_n`	Input	1	Active-low reset. Clears all state and outputs when asserted
`instr_valid`	Input	1	Indicates that fetch is presenting a new instruction this cycle
`instr`	Input	16	Instruction word: [15:12] opcode, [11:8] rd, [7:4] rs, [3:0] imm
`decode_done`	Output	1	Single-cycle pulse signaling decode completion for an instruction
`opcode`	Output	4	Decoded opcode field of the accepted instruction
`rd`	Output	4	Destination register index of the accepted instruction
`rs`	Output	4	Source register index of the accepted instruction
`imm`	Output	4	Immediate field of the accepted instruction
`hazard_stall`	Output	1	Data hazard that stalls acceptance of the current instruction

Below is the description of the behavioral requirements for this design:

- Instruction acceptance
  - An instruction is accepted when **`instr_valid`**=1 and **`hazard_stall`**=0 in the next cycle.
  - On acceptance, the fields (opcode, rd, rs, imm) must capture the corresponding bits of **`instr`**.
- Decode latency
  - After acceptance in cycle N, **`decode_done`** must assert exactly in cycle N+2.
  - **`decode_done`** must be a single cycle pulse.
- Hazard detection
  - A hazard occurs when the destination register (**`rd`**) of the previously accepted instruction matches the source register (**`rs`**) of the current instruction.
  - When a hazard is detected:
    - **`hazard_stall`** must assert.
    - The current instruction is not accepted until the stall clears.
    - **`decode_done`** must not assert during the stall window.
- Field stability
  - While **`instr_valid`**=1 and **`hazard_stall`**=1, the decoded fields (opcode, rd, rs, imm) must retain their previous values and only update when the instruction is accepted.
- Back-to-back instructions
  - If **`instr_valid`** is high in consecutive cycles and no hazard occurs, each instruction is accepted immediately.
  - Each accepted instruction produces a **`decode_done`** pulse two cycles later, possibly resulting in consecutive **`decode_done`** pulses.
- Reset behavior
  - When **`rst_n`**=0:
    - All outputs must be cleared to zero.
    - No instruction is accepted or completed until reset is released.

3.2. Testbench

The provided tb_decode_unit.sv testbench can be used as a basis for you to add the assertions. It contains test generation and result checking. You do not need to implement any other functionality, only focus on adding the assertions.

3.3. Assertions

Define what assertions are required for this design and test them with the given testbench.

As with previous module, perform the following tasks:

a) Try that all your assertions pass with the provided testbench and design.
b) Change the design (introduce a bug) and check that your assert/s fail.

4. What to Turn In

(2 pts) Deliver the pre-release after the 2h lab session

Please provide in a single PDF document:

1. Please indicate how many hours you spent on this lab. This will not affect your grade, but will be helpful for calibrating the workload for the future.
2. (4 pts) List of assertions for fetch_unit module.
3. (4 pts) List of assertions for decode_unit module.